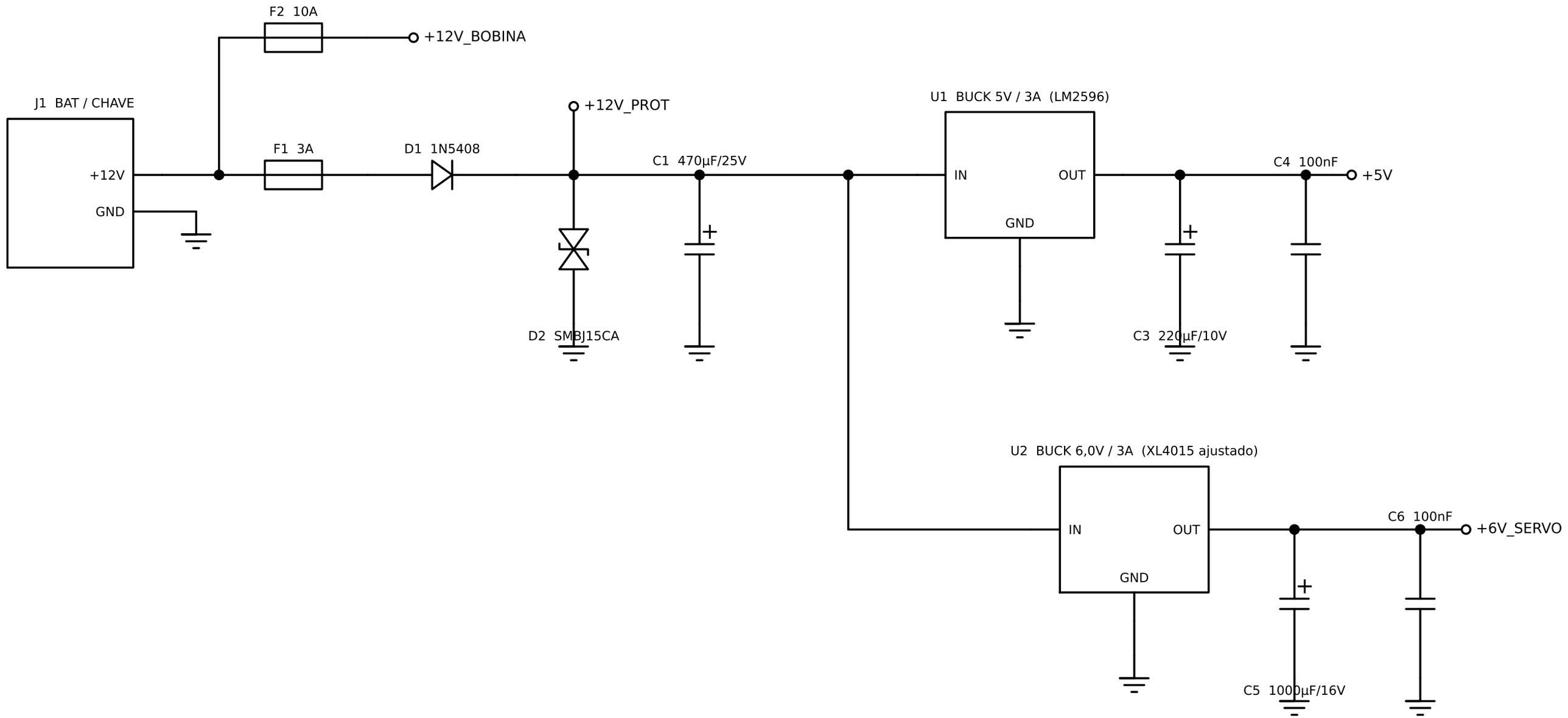
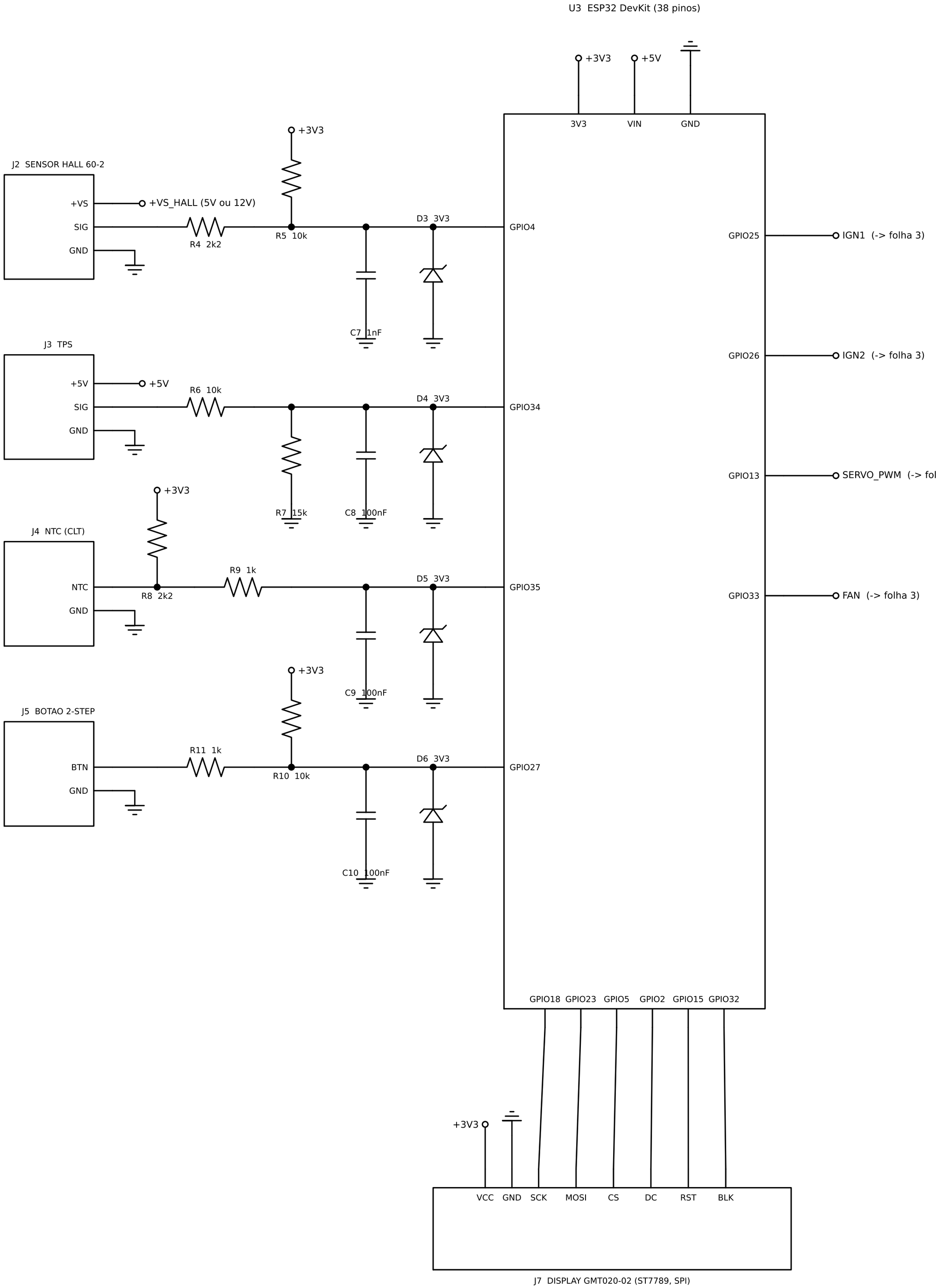
FOLHA 1/3 - ALIMENTAÇÃO E PROTEÇÃO

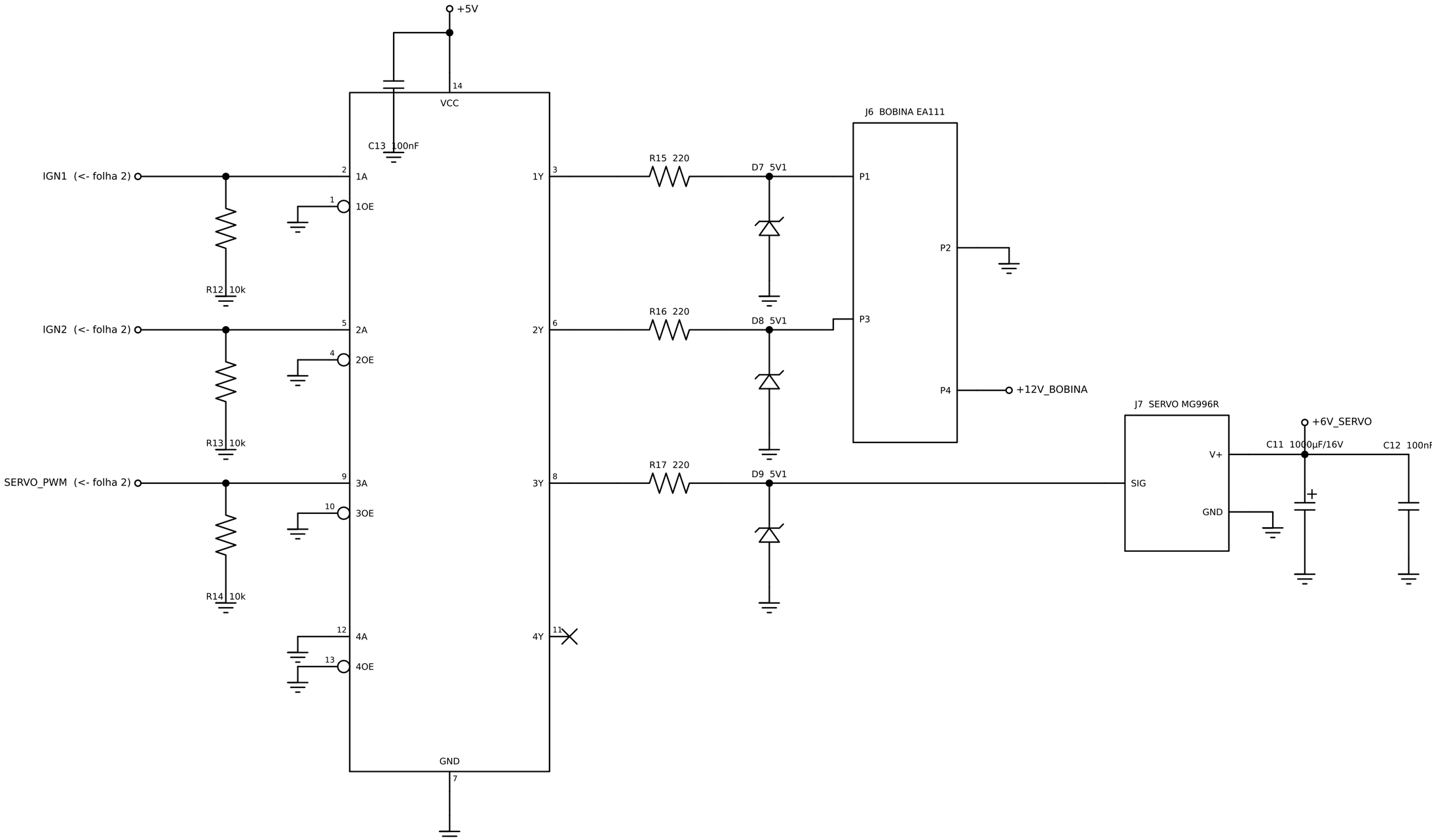


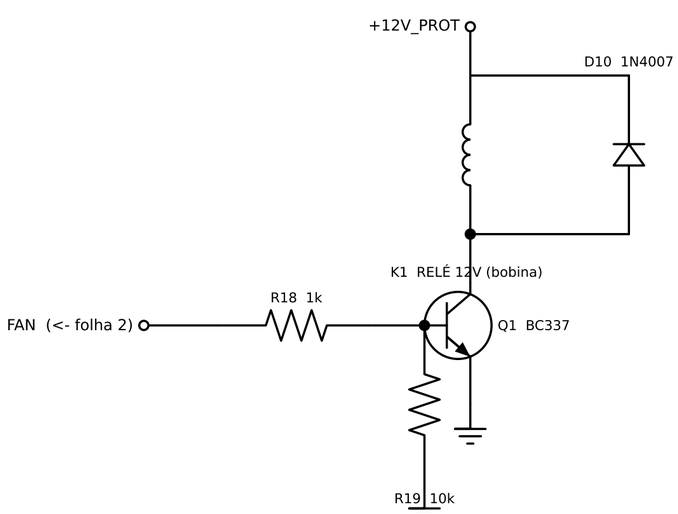
FOLHA 2/3 - ESP32, ENTRADAS DE SENSORES E DISPLAY



FOLHA 3/3 - SAÍDAS: BOBINAS, SERVO E VENTOINHA

U4 74HCT125 (buffer 3,3V -> 5V)





Pinagem e conexões - ECU ESP32 Chevette

Fonte: config.h, README.md, crank.cpp, ignition.cpp, fueling.cpp, thermal.cpp (branch main). Conectores J1 a J7 são os da folha 2 e 3.

GPIO	Função no código	Sentido	Condicionamento no esquema	Conector / destino
4	Sensor Hall 60-2 (ISR, borda de subida, INPUT_PULLUP)	Entrada	R4 2k2 série, R5 10k a 3V3, D3 zener 3V3, C7 1nF	J2
34	TPS (ADC1_CH6, 11 dB, só entrada)	Entrada analógica	R6 10k / R7 15k (5V vira 3,0V), D4 zener 3V3, C8 100nF	J3
35	CLT / NTC (ADC1_CH7, 11 dB, só entrada)	Entrada analógica	R8 2k2 a 3V3 (= rPullup 2200), R9 1k série, D5 zener 3V3, C9 100nF	J4
27	Botão 2-step (ativo em nível baixo, INPUT_PULLUP)	Entrada	R11 1k série, R10 10k a 3V3, D6 zener 3V3, C10 100nF	J5
25	Disparo bobina pino 1 (cil. 1+4), ativo-alto	Saída	U4 canal 1 (74HCT125), R15 220, D7 zener 5V1	J6 P1
26	Disparo bobina pino 3 (cil. 2+3), ativo-alto	Saída	U4 canal 2, R16 220, D8 zener 5V1	J6 P3
13	Servo MG996R, PWM 50 Hz, pulso 500-2400 us	Saída	U4 canal 3, R17 220, D9 zener 5V1	J7 SIG
33	Relé da ventoinha (ativo-alto)	Saída	R18 1k, Q1 BC337, R19 10k pull-down, D10 1N4007	K1
18 / 23	SPI SCK / MOSI do display	Saída	direto (3,3V nativo)	J7 display
5 / 2 / 15	CS / DC / RST do display (pinos de strapping)	Saída	direto	J7 display
32	Backlight (BLK), nível alto = ligado	Saída	direto (ver nota 8)	J7 display

Lista de materiais

Ref.	Valor / modelo	Descrição	Qtd
U3	ESP32 DevKit 38 pinos	Microcontrolador	1
U4	74HCT125 (DIP-14)	Buffer quádruplo, entrada compatível 3,3V, saída 5V	1
U1	Módulo buck LM2596	Ajustar para 5,0V antes de ligar o ESP32	1
U2	Módulo buck XL4015 (ou equivalente 3A)	Ajustar para 6,0V antes de ligar o servo	1
Q1	BC337	NPN, aciona bobina do relé	1
D1	1N5408	Proteção contra inversão de polaridade	1
D2	SMBJ15CA	TVS bidirecional 15V, proteção de surto	1
D3, D4, D5, D6	BZX55C3V3	Zener 3,3V 0,5W, grampo das entradas	4
D7, D8, D9	BZX55C5V1	Zener 5,1V 0,5W, grampo das saídas 5V	3
D10	1N4007	Diodo de roda livre do relé	1
R4, R8	2k2 1/4W	Série do Hall; pull-up do NTC	2
R5, R6, R10, R12, R13, R14, R19	10k 1/4W	Pull-up, divisor, pull-downs	7
R7	15k 1/4W	Divisor do TPS	1
R9, R11, R18	1k 1/4W	Séries (NTC, botão) e base do Q1	3
R15, R16, R17	220 ohm 1/4W	Série das saídas dos buffers	3
C1	470 uF / 25V	Eletrolítico, entrada 12V	1
C3	220 uF / 10V	Eletrolítico, saída 5V	1
C5, C11	1000 uF / 16V	Eletrolítico, alimentação do servo (um no buck, um no conector)	2
C7	1 nF	Filtro do Hall	1
C4, C6, C8, C9, C10, C12, C13	100 nF	Desacoplamento e filtros	7
F1 / F2	3A / 10A	Fusíveis: ECU e bobinas	1 / 1
K1	Relé automotivo 12V 30A	Ventoinha, contatos com fusível próprio fora da ECU	1
J1 a J7	Bornes / conectores	Bateria, Hall, TPS, NTC, botão, bobina EA111, servo, display	7

Notas e achados na leitura do código

1. Servo

Confirmado: MG996R de 180 graus, posicional (não é de rotação contínua). fueling.cpp usa ESP32Servo, setPeriodHertz(50), attach(pino 13, 500, 2400), write(0 a 180). Padrão de partida 10 graus (fechado) e 170 graus (aberto). Algumas unidades MG996R só dão cerca de 120 graus reais: teste no modo de teste do webapp e ajuste servoAngleAtClosedThrottle e servoAngleAtFullThrottle.

2. Diagrama de pinagem

O repositório tem tabela de pinagem (README, seção 1) e os defines do config.h. Não existe diagrama gráfico nem esquema elétrico: este documento é o primeiro.

3. Polaridade do disparo

ignition.cpp aciona o GPIO em nível ALTO para iniciar o dwell e em nível BAIXO para disparar a faísca; em repouso e no corte tudo fica em BAIXO. O circuito do README (NPN com coletor na bobina) inverte o sinal: a bobina ficaria carregando em repouso. Por isso o esquema usa buffer não inversor (74HCT125). Confirme com osciloscópio antes de ligar a EA111.

4. Bug em crank.cpp

A ISR descarta qualquer período de dente menor que 300 us. Numa roda 60-2, 6000 rpm geram 6000 dentes por segundo (166 us por dente), então o filtro descarta todos os dentes acima de cerca de 3300 rpm. O comentário do código diz 1000 us a 10000 rpm, mas o valor correto é 100 us. Sugestão: trocar o limite por algo na faixa de 30 a 40 us. O filtro de hardware (R4 com C7, constante de 2,2 us) passa 10 kHz sem problema.

5. Hall

A ISR usa borda de subida e INPUT_PULLUP. O sentido da borda depende do sensor; o ajuste fino fica em triggerFineOffsetDeg (lâmpada estroboscópica). Alimente o sensor conforme o datasheet (5V ou 12V); R4 e o zener D3 aceitam saída de 5V ou 12V.

6. TPS e NTC

Divisor do TPS: 5V vira 3,0V, o zener de 3,3V fica sem conduzir em operação normal. Refaça a calibração pelo webapp (padrão tpsAdcMin 200 e tpsAdcMax 3800). NTC: R8 2k2 é o rPullup padrão (2200) e fica no lado do conector; R9 não entra na conta porque o ADC é de alta impedância.

7. Estado no boot

Os GPIO 25, 26, 13 e 33 ficam flutuando no reset. Os pull-downs R12, R13, R14 e R19 e os pinos OE do 74HCT125 em GND mantêm as saídas em nível baixo até o firmware assumir.

8. Display

GPIO 2, 5 e 15 são pinos de strapping. Se houver bootloop, desconecte o display. O pino BLK (GPIO32) é saída digital direta: se o seu módulo ligar o BLK direto ao LED sem transistor, acione-o via transistor (corrente do GPIO limitada a cerca de 12 mA).

9. Alimentação

O servo usa buck próprio de 6V/3A (pico de estol acima de 2A) com GND comum. A bobina EA111 recebe 12V direto pelo fusível F2, não pelos bucks. Pinos 2 e 4 da bobina (GND e +12V) precisam ser confirmados com multímetro e o manual da bobina.

10. Limites desta entrega

O esquema foi desenhado com a biblioteca schemdraw (símbolos padrão, designadores, valores), não com KiCad. Não foi simulado nem validado em bancada. Valores de zener e divisores foram calculados para as faixas descritas nas notas.